Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Навчально-науковий інститут комп’ютерних систем
Кафедра інформаційних систем

Лабораторна робота № 6
з дисципліни: «Теорія алгоритмів»
Тема: «Алгоритми на графах. Пошук найкоротшого шляху»

Варіант № 15

Виконав:
Студент групи АІ-254
Панькевич В. В.
Перевірили:
Смик С. Ю.
Арсірій О.О.

Одеса 2026

Мета роботи: набуття практичних навичок із проєктування, реалізації, тестування та аналізу алгоритмів Дейкстри та Флойда для пошуку найкоротших шляхів між парами вершин.

Завдання:

1. Побудувати за допомогою алгоритму Дейкстри найкоротший шлях із заданої вершини графа за варіантами завдань заповнити таблиці 6.1 та 6.2

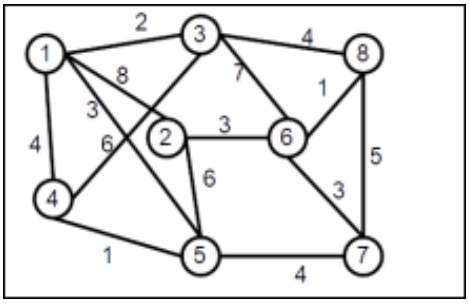
2. Показати графічно хід побудови найкоротшого шляху за алгоритмом Дейкстри, використовуючи процедуру Dijkstra(G, s)

3. Запрограмувати алгоритм Дейкстри. Запустити програму, показати результат за варіантами завдань. Показати, що результати ручної та автоматизованої побудови найкоротшего шляху співпадають

4. Показати хід побудови матриці відстаней за алгоритмом Флойда, використовуючи процедуру Floyd (W [l..n, l..n]). Заповнити таблицю 6.3.

5.Запрограмувати алгоритм Флойда. Запустити програму, показати результат за варіантами завдань. Показати, що результати ручної та автоматизованої побудови матриць відстаней співпадають

6. Порівняти між собою результати моделювання за алгоритмом Дейкстри та Флойда

Варіант	Граф
15	

Результати виконання завдання:

1. Вихідні дані та представлення графа

Для виконання лабораторної роботи використовується граф варіанта №15, представлений на рисунку 6.1. Граф є неорієнтованим, зваженим і зв'язним, містить 8 вершин та 14 ребер із додатними вагами.

Для пошуку найкоротших шляхів використовуватимуться алгоритми Дейкстри (Dijkstra) та Флойда (Floyd–Warshall). Алгоритм Дейкстри дозволяє визначити найкоротші шляхи від заданої початкової вершини до всіх інших, тоді як алгоритм Флойда знаходить найкоротші відстані між усіма парами вершин графа.

Для алгоритму Дейкстри початковою обрано вершину 1. Для реалізації алгоритму Флойда граф буде представлено у вигляді матриці ваг, яка використовується як початкова матриця відстаней.

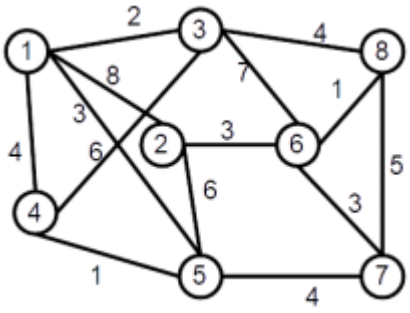


Рисунок 6.1 – Зважений неорієнтований граф варіанта №15

1.1. Представлення графа списком ребер

Для подальших розрахунків використано такі ребра:

№	Ребро	Вага		№	Ребро	Вага
1	(1, 2)	8		8	(3, 6)	7
2	(1, 3)	2		9	(3, 8)	4
3	(1, 4)	4		10	(4, 5)	1
4	(1, 5)	3		11	(5, 7)	4

5	(2, 5)	6		12	(6, 7)	3
6	(2, 6)	3		13	(6, 8)	1
7	(3, 4)	6		14	(7, 8)	5

1.2. Побудова матриці ваг графа

Для представлення зваженого графа побудовано матрицю ваг W розміром 8 × 8. Елемент w_{ij} визначає вагу ребра між вершинами i та j. Якщо вершини безпосередньо не з'єднані, відповідному елементу матриці присвоюється значення ∞. На головній діагоналі розміщуються нулі, оскільки відстань від вершини до самої себе дорівнює нулю.

Оскільки граф є неорієнтованим, матриця ваг симетрична відносно головної діагоналі. Отримана матриця використовуватиметься для реалізації алгоритмів Дейкстри та Флойда.

Початкова матриця ваг $W=D^{(0)}$:

i \ j	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
1	0	8	2	4	3	∞	∞	∞
2	8	0	∞	∞	6	3	∞	∞
3	2	∞	0	6	∞	7	∞	4
4	4	∞	6	0	1	∞	∞	∞
5	3	6	∞	1	0	∞	4	∞
6	∞	3	7	∞	∞	0	3	1
7	∞	∞	∞	∞	4	3	0	5
8	∞	∞	4	∞	∞	1	5	0

1.3. Підготовка початкових даних для алгоритму Дейкстри

- dist[] – найкоротші відомі відстані від початкової вершини;
- pred[] – попередники вершин у знайдених шляхах.

На початку алгоритму для $s = 1$ маємо:

$$dist = [0, \infty, \infty, \infty, \infty, \infty, \infty]$$

$$pred = [-1, -1, -1, -1, -1, -1, -1, -1]$$

Нуль відповідає початковій вершині 1. Для решти вершин відстані поки невідомі.

Таким чином, для графа визначено множину вершин і зважених ребер, побудовано матрицю ваг та виконано ініціалізацію масивів відстаней і попередників для початкової вершини 1. Отримані дані будуть використані для ручного трасування алгоритму Дейкстри, побудови матриць відстаней за алгоритмом Флойда та перевірки результатів за допомогою програмних реалізацій мовою Python.

2. Побудова найкоротших шляхів за алгоритмом Дейкстри

Алгоритм Дейкстри (Dijkstra) використовується для знаходження найкоротших шляхів від заданої початкової вершини до всіх інших вершин зваженого графа з невід'ємними вагами ребер.

Алгоритм починає роботу з ініціалізації масивів відстаней dist та попередників pred. На кожному кроці з черги з пріоритетом pq обирається вершина з найменшою поточною відстанню, після чого перевіряється можливість скорочення шляхів до її суміжних вершин шляхом релаксації ребер.

Для реалізації алгоритму використовується граф варіанта №15, представлений на рисунку 6.1. Початковою обрано вершину 1. У разі однакових мінімальних відстаней перевага надається вершині з меншим номером. Значення попередника змінюється лише тоді, коли знайдено строго коротший шлях.

Псевдокод алгоритму Дейкстри наведено в лістингу 6.1.

Лістинг 6.1 – Псевдокод алгоритму Дейкстри

```
Dijkstra(G, s)

// Вхідні дані: зважений граф G = (V, E), початкова вершина s
// Вихідні дані: масиви pred і dist

pred ← CreateArray(|V|)
dist ← CreateArray(|V|)
pq ← CreatePQ()
```

```
// Ініціалізація масивів
for each v ∈ V do
    pred[v] ← -1

    if v ≠ s then
        dist[v] ← ∞
    else
        dist[v] ← 0

    // Додаємо вершину до черги
    InsertInPQ(pq, v, dist[v])

// Пошук найкоротших шляхів
while SizePQ(pq) ≠ 0 do

    // Обираємо вершину з найменшою відстанню
    u ← ExtractMinFromPQ(pq)

    // Переглядаємо суміжні вершини
    for each v ∈ AdjacencyList(G, u) do

        // Перевіряємо можливість релаксації
        if dist[v] > dist[u] + Weight(G, u, v) then

            // Оновлюємо найкоротшу відстань
            dist[v] ← dist[u] + Weight(G, u, v)

            // Зберігаємо попередника
            pred[v] ← u

            // Оновлюємо пріоритет вершини
            UpdatePQ(pq, v, dist[v])

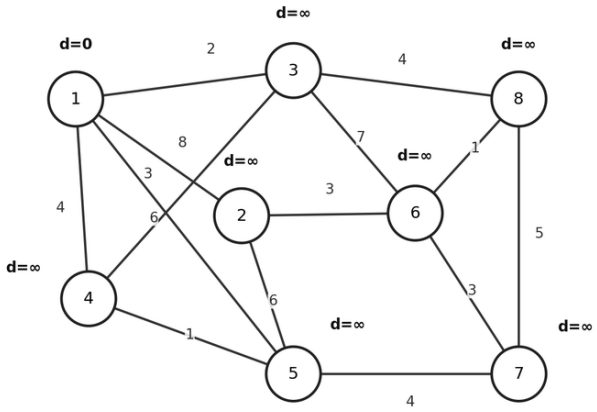
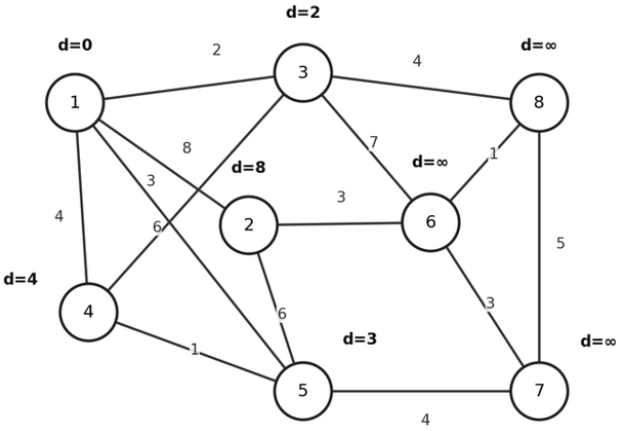
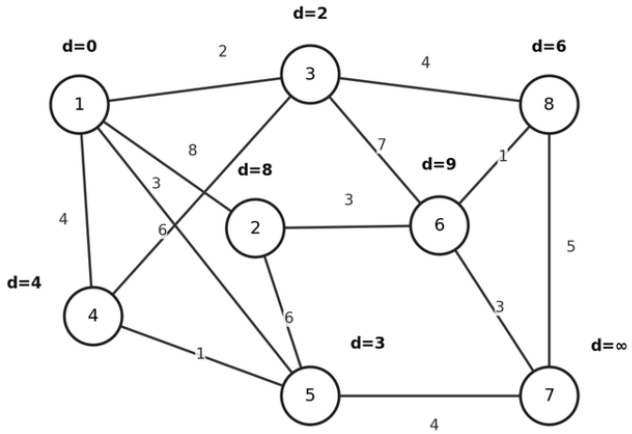
return (pred, dist)
```

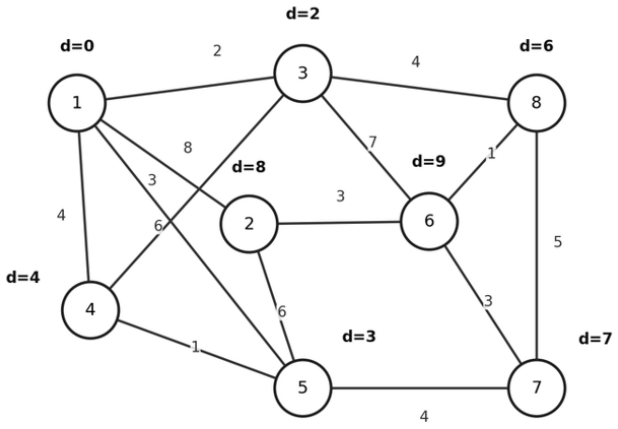
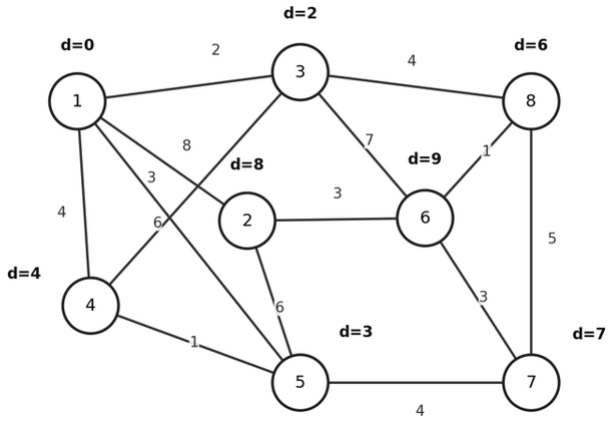
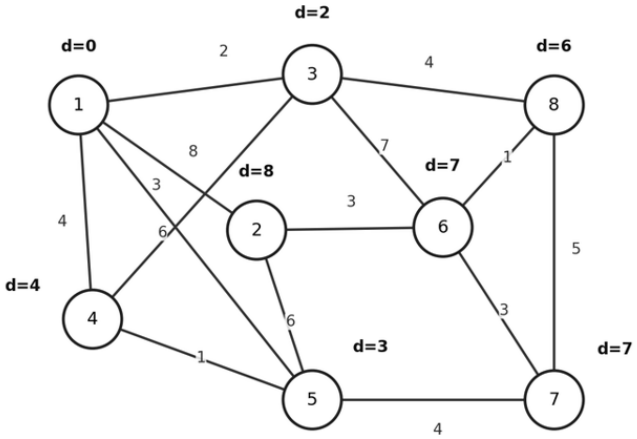
2.1. Ручне трасування алгоритму Дейкстри

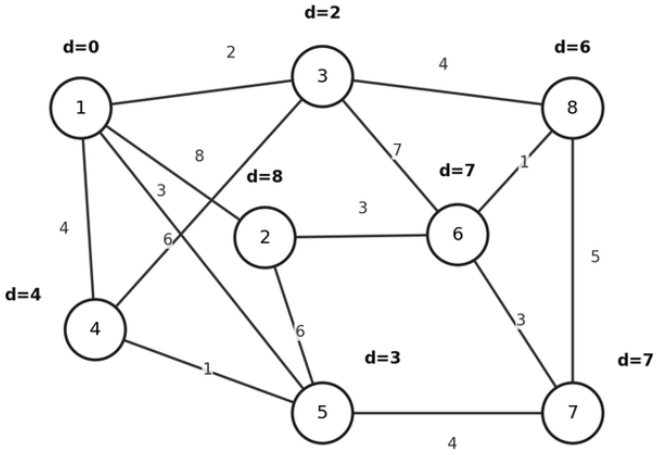
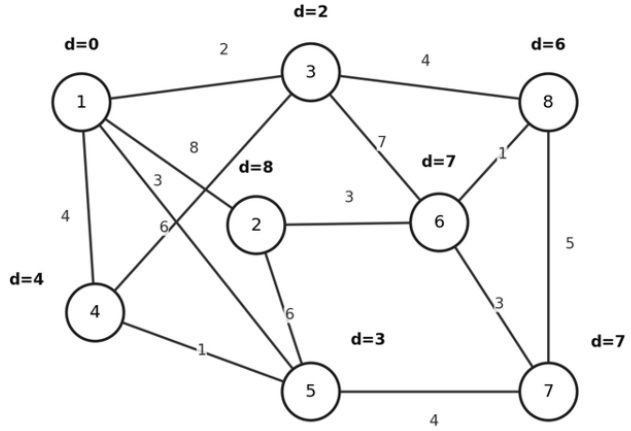
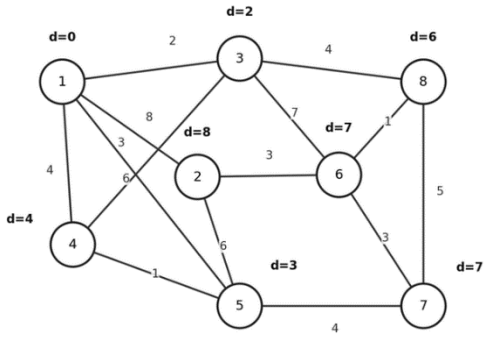
Для дослідження роботи алгоритму Дейкстри виконано ручне трасування пошуку найкоротших шляхів від початкової вершини 1. На кожному кроці зафіксовано поточну вершину, результати релаксації ребер, зміни масивів відстаней dist і попередників pred, а також стан черги з пріоритетом pq.

Для наочного представлення процесу побудови найкоротших шляхів використано графічні схеми, на яких збережено початкове розташування вершин і ребер. На кожному кроці біля вершин відображено поточні значення найкоротших відомих відстаней, які змінюються внаслідок релаксації ребер. Результати трасування наведено в таблиці 6.1.

Таблиця 6.1 – Хід побудови найкоротших шляхів за алгоритмом Дейкстри

Зображення	Хід виконання алгоритму
	**Ініціалізація (s = 1)** dist = [0, ∞, ∞, ∞, ∞, ∞, ∞, ∞] pred = [-1, -1, -1, -1, -1, -1, -1, -1] pq = {(1,0),(2,∞),(3,∞),(4,∞), (5,∞),(6,∞),(7,∞),(8,∞)} **Крок 1. Обробка вершини 1 dist = 0** **Суміжні вершини: 2, 3, 4, 5.** **Релаксація ребер:** dist[2]: ∞ > 0 + 8 = 8 → 8, pred[2] = 1 dist[3]: ∞ > 0 + 2 = 2 → 2, pred[3] = 1 dist[4]: ∞ > 0 + 4 = 4 → 4, pred[4] = 1 dist[5]: ∞ > 0 + 3 = 3 → 3, pred[5] = 1 **Поточний стан:** dist = [0, 8, 2, 4, 3, ∞, ∞, ∞] pred = [-1, 1, 1, 1, 1, -1, -1, -1] pq = {(3,2),(5,3),(4,4),(2,8), (6,∞),(7,∞),(8,∞)}
	
	**Крок 2. Обробка вершини 3 dist = 2** **Суміжні вершини: 1 (оброблена), 4, 6, 8.** **Релаксація ребер:** dist[4]: 4 > 2 + 6 = 8 → ні dist[6]: ∞ > 2 + 7 = 9 → 9, pred[6] = 3 dist[8]: ∞ > 2 + 4 = 6 → 6, pred[8] = 3 **Поточний стан:** dist = [0, 8, 2, 4, 3, 9, ∞, 6] pred = [-1, 1, 1, 1, 1, 3, -1, 3] pq = {(5,3),(4,4),(8,6),(2,8),(6,9),(7,∞)}

	**Крок 3. Обробка вершини 5 dist = 3** **Суміжні вершини: 1 (оброблена), 2, 4, 7.** **Релаксація ребер:** dist[2]: 8 > 3 + 6 = 9 → ні dist[4]: 4 > 3 + 1 = 4 → ні dist[7]: ∞ > 3 + 4 = 7 → 7, pred[7] = 5 Для вершини 4 знайдено шлях однакової довжини 4, тому попередник залишається 1. **Поточний стан:** dist = [0, 8, 2, 4, 3, 9, 7, 6] pred = [-1, 1, 1, 1, 1, 3, 5, 3] pq = {(4,4),(8,6),(7,7),(2,8),(6,9)}
	**Крок 4. Обробка вершини 4 dist = 4** **Суміжні вершини: 1, 3, 5 - усі вже оброблені. Оновлень немає.** **Поточний стан:** dist = [0, 8, 2, 4, 3, 9, 7, 6] pred = [-1, 1, 1, 1, 1, 3, 5, 3] pq = {(8,6),(7,7),(2,8),(6,9)}
	**Крок 5. Обробка вершини 8 dist = 6** **Суміжні вершини: 3 (оброблена), 6, 7.** **Релаксація ребер:** dist[6]: 9 > 6 + 1 = 7 → 7, pred[6] = 8 dist[7]: 7 > 6 + 5 = 11 → ні **Поточний стан:** dist = [0, 8, 2, 4, 3, 7, 7, 6] pred = [-1, 1, 1, 1, 1, 8, 5, 3] pq = {(6,7),(7,7),(2,8)}

	**Крок 6. Обробка вершини 6 dist = 7** **Суміжні вершини: 2, 3, 7, 8. Вершини 3 та 8 уже оброблені.** **Релаксація ребер:** dist[2]: 8 > 7 + 3 = 10 → ні dist[7]: 7 > 7 + 3 = 10 → ні **Поточний стан:** dist = [0, 8, 2, 4, 3, 7, 7, 6] pred = [-1, 1, 1, 1, 1, 8, 5, 3] pq = {(7,7),(2,8)}
	**Крок 7. Обробка вершини 7 dist = 7** Суміжні вершини: 5, 6, 8 - усі вже оброблені. Оновлень немає. **Поточний стан:** dist = [0, 8, 2, 4, 3, 7, 7, 6] pred = [-1, 1, 1, 1, 1, 8, 5, 3] pq = {(2,8)}
	**Крок 8. Обробка вершини 2 dist = 8** **Суміжні вершини: 1, 5, 6 - усі вже оброблені. Оновлень немає.** **Кінцевий стан:** dist = [0, 8, 2, 4, 3, 7, 7, 6] pred = [-1, 1, 1, 1, 1, 8, 5, 3] pq = {}

2.2. Визначення найкоротших шляхів

У результаті ручного виконання алгоритму Дейкстри було визначено найкоротші відстані від початкової вершини 1 до всіх інших вершин графа.

За допомогою масиву попередників pred відновлено відповідні найкоротші шляхи. Результати наведено в таблиці 6.2.

Таблиця 6.2 – Найкоротші шляхи від вершини 1 до решти вершин графа

З вершини	У вершину	Найкоротший шлях	Вага
1	2	1 → 2	8
	3	1 → 3	2
	4	1 → 4	4
	5	1 → 5	3
	6	1 → 3 → 8 → 6	7
	7	1 → 5 → 7	7
	8	1 → 3 → 8	6

Аналіз результатів:

За результатами ручного трасування алгоритму Дейкстри (таблиця 6.1) визначено найкоротші шляхи від початкової вершини 1 до всіх інших вершин графа (таблиця 6.2).

Найменшу відстань отримано до вершини 3, яка становить 2, а найбільшу – до вершини 2, яка становить 8. Під час виконання алгоритму відбувалося поступове оновлення масивів відстаней і попередників за допомогою релаксації ребер.

Зокрема, для вершини 6 початкова знайдена відстань 9 через вершину 3 була скорочена до 7 після обробки вершини 8. Це дозволило отримати найкоротший шлях 1 → 3 → 8 → 6.

Таким чином, алгоритм Дейкстри дозволив визначити найкоротші шляхи від початкової вершини до всіх інших вершин зваженого графа.

3. Програмна реалізація алгоритму Дейкстри

Для перевірки результатів ручного трасування алгоритму Дейкстри було розроблено програмну реалізацію мовою Python. Як вхідні дані використано зважений неорієнтований граф, який містить 8 вершин і 14 ребер. Початковою вершиною обрано вершину 1.

Програма реалізує пошук найкоротших шляхів із використанням черги з пріоритетом, масиву відстаней dist та масиву попередників pred. На кожному кроці відбувається вибір вершини з найменшою відстанню, перевірка суміжних вершин і релаксація відповідних ребер.

Для забезпечення відповідності ручному трасуванню в разі однакових відстаней перевага надається вершині з меншим номером, а попередники змінюються лише за умови знаходження строго коротшого шляху.

Python-код програмної реалізації алгоритму Дейкстри наведено в Лістингу 6.2.

Лістинг 6.2 – Python-код реалізації алгоритму Дейкстри

```
import heapq

# Кількість вершин графа
N = 8

# Ребра графа варіанта №15 (u, v, вага)
EDGES = [
    (1, 2, 8), (1, 3, 2), (1, 4, 4), (1, 5, 3),
    (2, 5, 6), (2, 6, 3), (3, 4, 6), (3, 6, 7),
    (3, 8, 4), (4, 5, 1), (5, 7, 4), (6, 7, 3),
    (6, 8, 1), (7, 8, 5)
]

def build_graph(n, edges):
    """Побудова списку суміжності графа."""
    graph = {v: [] for v in range(1, n + 1)}

    # Граф неорієнтований
    for u, v, weight in edges:
        graph[u].append((v, weight))
        graph[v].append((u, weight))

    # Сортуємо сусідів за номерами вершин
    for v in graph:
        graph[v].sort()

    return graph

def fmt(x):
    """Відображення нескінченності."""
    return "∞" if x == float("inf") else str(x)

def show_array(array):
    """Виведення масиву для вершин 1...8."""
    return "[" + ", ".join(map(fmt, array[1:])) + "]"

def show_queue(dist, visited):
    """Виведення актуального стану черги."""
    queue = sorted(
        (dist[v], v)
        for v in range(1, len(dist))
```

```
        if not visited[v]
    )

    return "{" + ", ".join(
        f"({v},{fmt(d)})" for d, v in queue
    ) + "}"

def reconstruct_path(pred, source, target):
    """Відновлення найкоротшого шляху."""
    path = []
    v = target

    while v != -1:
        path.append(v)

        if v == source:
            return path[::-1]
        v = pred[v]

    return []

def dijkstra(graph, start):
    """
    Алгоритм Дейкстри.
    Вхідні дані: граф, початкова вершина.
    Вихідні дані: dist, pred.
    """
    n = len(graph)
    inf = float("inf")

    # Ініціалізація масивів
    dist = [inf] * (n + 1)
    pred = [-1] * (n + 1)
    visited = [False] * (n + 1)
    dist[start] = 0

    # Створення черги з пріоритетом
    pq = [(dist[v], v) for v in range(1, n + 1)]
    heapq.heapify(pq)

    print("--- Ініціалізація ---")
    print("dist =", show_array(dist))
    print("pred =", show_array(pred))
    print("pq =", show_queue(dist, visited))

    step = 0

    # Основний цикл алгоритму
    while pq:
        d, u = heapq.heappop(pq)

        # Пропускаємо застарілі записи
        if visited[u] or d != dist[u]:
            continue

        if d == inf:
            break

        # Позначаємо вершину обробленою
        visited[u] = True
        step += 1

        print(f"\nКрок {step}. "
              f"Обробка вершини {u}; dist = {fmt(d)}")
```

```
            # Переглядаємо суміжні вершини
            for v, weight in graph[u]:

                if visited[v]:
                    continue

                # Обчислюємо нову відстань
                new_d = dist[u] + weight

                # Перевірка можливості релаксації
                if new_d < dist[v]:
                    print(
                        f"  ({u},{v}): {fmt(dist[v])} > "
                        f"{fmt(dist[u])} + {weight} = "
                        f"{fmt(new_d)} → оновлення"
                    )

                    # Оновлення відстані та попередника
                    dist[v] = new_d
                    pred[v] = u

                    # Додавання нового пріоритету
                    heapq.heappush(pq, (new_d, v))

                else:
                    print(
                        f"  ({u},{v}): {fmt(dist[v])} > "
                        f"{fmt(dist[u])} + {weight} = "
                        f"{fmt(new_d)} → ні"
                    )

            # Поточний стан алгоритму
            print("dist =", show_array(dist))
            print("pred =", show_array(pred))
            print("pq =", show_queue(dist, visited))
        return dist, pred

    # Основна частина програми
    graph = build_graph(N, EDGES)

    # Початкова вершина
    start = 1
    dist, pred = dijkstra(graph, start)

    # Виведення найкоротших шляхів
    print("\n--- Найкоротші шляхи ---")

    for v in range(1, N + 1):
        if v == start:
            continue

        path = reconstruct_path(pred, start, v)
        print(
            f"{start} → {v}: "
            + " → ".join(map(str, path))
            + f" | вага = {fmt(dist[v])}"
        )

    # Підсумкові результати
    print("\n --- Кінцеві масиви ---")
    print("dist =", show_array(dist))
    print("pred =", show_array(pred))
```

У результаті виконання Python-коду, наведеного в Лістингу 6.2, було визначено найкоротші шляхи від початкової вершини 1 до всіх інших вершин графа варіанта №15.

Програма виконала вісім кроків алгоритму Дейкстри, послідовно обробляючи вершини та здійснюючи релаксацію ребер. На кожному кроці було зафіксовано поточні значення масивів відстаней dist, попередників pred і стан черги з пріоритетом pq.

Стан черги pq у програмному виводі представлено як упорядкований перелік необроблених вершин з актуальними відстанями, що дозволяє наочно зіставити результати програмного виконання з ручним трасуванням.

Результати програмного виконання алгоритму Дейкстри наведено в таблиці 6.3.

Таблиця 6.3 – Результати виконання алгоритму Дейкстри мовою Python

```
--- Ініціалізація ---
dist = [0, ∞, ∞, ∞, ∞, ∞, ∞, ∞]
pred = [-1, -1, -1, -1, -1, -1, -1, -1]
pq = {(1,0), (2,∞), (3,∞), (4,∞), (5,∞), (6,∞), (7,∞), (8,∞)}

Крок 1. Обробка вершини 1; dist = 0
  (1,2): ∞ > 0 + 8 = 8 → оновлення
  (1,3): ∞ > 0 + 2 = 2 → оновлення
  (1,4): ∞ > 0 + 4 = 4 → оновлення
  (1,5): ∞ > 0 + 3 = 3 → оновлення
dist = [0, 8, 2, 4, 3, ∞, ∞, ∞]
pred = [-1, 1, 1, 1, 1, -1, -1, -1]
pq = {(3,2), (5,3), (4,4), (2,8), (6,∞), (7,∞), (8,∞)}

Крок 2. Обробка вершини 3; dist = 2
  (3,4): 4 > 2 + 6 = 8 → ні
  (3,6): ∞ > 2 + 7 = 9 → оновлення
  (3,8): ∞ > 2 + 4 = 6 → оновлення
dist = [0, 8, 2, 4, 3, 9, ∞, 6]
pred = [-1, 1, 1, 1, 1, 3, -1, 3]
pq = {(5,3), (4,4), (8,6), (2,8), (6,9), (7,∞)}

Крок 3. Обробка вершини 5; dist = 3
  (5,2): 8 > 3 + 6 = 9 → ні
  (5,4): 4 > 3 + 1 = 4 → ні
  (5,7): ∞ > 3 + 4 = 7 → оновлення
dist = [0, 8, 2, 4, 3, 9, 7, 6]
pred = [-1, 1, 1, 1, 1, 3, 5, 3]
pq = {(4,4), (8,6), (7,7), (2,8), (6,9)}

Крок 4. Обробка вершини 4; dist = 4
dist = [0, 8, 2, 4, 3, 9, 7, 6]
pred = [-1, 1, 1, 1, 1, 3, 5, 3]
pq = {(8,6), (7,7), (2,8), (6,9)}
```

```
Крок 5. Обробка вершини 8; dist = 6
  (8,6): 9 > 6 + 1 = 7 → оновлення
  (8,7): 7 > 6 + 5 = 11 → ні
dist = [0, 8, 2, 4, 3, 7, 7, 6]
pred = [-1, 1, 1, 1, 1, 8, 5, 3]
pq = {(6,7), (7,7), (2,8)}

Крок 6. Обробка вершини 6; dist = 7
  (6,2): 8 > 7 + 3 = 10 → ні
  (6,7): 7 > 7 + 3 = 10 → ні
dist = [0, 8, 2, 4, 3, 7, 7, 6]
pred = [-1, 1, 1, 1, 1, 8, 5, 3]
pq = {(7,7), (2,8)}

Крок 7. Обробка вершини 7; dist = 7
dist = [0, 8, 2, 4, 3, 7, 7, 6]
pred = [-1, 1, 1, 1, 1, 8, 5, 3]
pq = {(2,8)}

Крок 8. Обробка вершини 2; dist = 8
dist = [0, 8, 2, 4, 3, 7, 7, 6]
pred = [-1, 1, 1, 1, 1, 8, 5, 3]
pq = {}

--- Найкоротші шляхи ---
1 → 2: 1 → 2 | вага = 8
1 → 3: 1 → 3 | вага = 2
1 → 4: 1 → 4 | вага = 4
1 → 5: 1 → 5 | вага = 3
1 → 6: 1 → 3 → 8 → 6 | вага = 7
1 → 7: 1 → 5 → 7 | вага = 7
1 → 8: 1 → 3 → 8 | вага = 6

--- Кінцеві масиви ---
dist = [0, 8, 2, 4, 3, 7, 7, 6]
pred = [-1, 1, 1, 1, 1, 8, 5, 3]
```

Аналіз результатів:

Порівняння результатів ручного трасування алгоритму Дейкстри (таблиця 6.1) та програмного виконання (таблиця 6.3) показало їх повний збіг. Послідовність оброблення вершин, результати релаксації ребер і кінцеві значення масивів dist та pred відповідають ручним розрахункам.

У результаті виконання програми отримано масив найкоротших відстаней dist = [0, 8, 2, 4, 3, 7, 7, 6] і масив попередників pred = [-1, 1, 1, 1, 1, 8, 5, 3].

Отримані найкоротші шляхи також відповідають результатам таблиці 6.2. Зокрема, найкоротший шлях від вершини 1 до вершини 6 проходить через вершини 3 та 8 і має загальну вагу 7.

Таким чином, програмна реалізація алгоритму Дейкстри підтвердила правильність ручної побудови найкоротших шляхів для графа.

4. Побудова матриці найкоротших відстаней за алгоритмом Флойда

Алгоритм Флойда (Floyd–Warshall) використовується для визначення найкоротших відстаней між усіма парами вершин зваженого графа. На відміну від алгоритму Дейкстри, який знаходить найкоротші шляхи від однієї початкової вершини, алгоритм Флойда дозволяє отримати матрицю найкоротших відстаней між будь-якими двома вершинами.

Основою алгоритму є послідовне уточнення матриці відстаней шляхом перевірки можливості скорочення маршрутів через проміжні вершини. На кожній ітерації розглядається чергова вершина k, через яку може проходити коротший шлях між вершинами i та j.

Для виконання алгоритму використовується граф, представлений на рисунку 6.1, та початкова матриця ваг $W = D^{(0)}$, побудована в підрозділі 1.2. Оскільки граф містить 8 вершин, необхідно сформувати послідовність матриць від $D^{(0)}$ до $D^{(8)}$.

Псевдокод алгоритму Флойда наведено в Лістингу 6.3.

Лістинг 6.3 – Псевдокод алгоритму Флойда

```
Floyd(W)

// Вхідні дані: матриця ваг W графа G = (V, E)
// Вихідні дані: матриця найкоротших відстаней D

n ← |V|
D ← Copy(W)

// Послідовно розглядаємо проміжні вершини
for k ← 1 to n do

    // Перебираємо початкові вершини
    for i ← 1 to n do

        // Перебираємо кінцеві вершини
        for j ← 1 to n do

            // Перевіряємо шлях через вершину k
            if D[i,j] > D[i,k] + D[k,j] then

                // Оновлюємо найкоротшу відстань
                D[i,j] ← D[i,k] + D[k,j]

return D
```

4.1. Ручне трасування алгоритму Флойда

Для дослідження роботи алгоритму Флойда виконано ручне трасування побудови матриці найкоротших відстаней між усіма парами вершин графа.

На початковому етапі використовується матриця ваг $D^{(0)}$. На кожній наступній ітерації перевіряється можливість скорочення відстаней через чергову проміжну вершину k. Якщо знайдено коротший маршрут, відповідний елемент матриці оновлюється.

Результати послідовного виконання алгоритму Флойда наведено в таблиці 6.4. Для наочності на кожному кроці показано матрицю відстаней та пояснення змін її елементів.

Для компактного представлення результатів у таблиці 6.4 наведено обчислення для тих пар вершин, відстані між якими скоротилися. Перевірки, що не призвели до оновлення елементів матриці, окремо не відображено.

Таблиця 6.4 – Хід побудови матриці найкоротших відстаней за алгоритмом Флойда

Проміжна вершина та розрахунки	Матриця відстаней
Ініціалізація: D(0) = W W ─ початкова матриця ваг графа варіанта №15. На головній діагоналі ─ 0; за відсутності безпосереднього ребра ─ ∞. Наступні кроки враховують проміжні вершини 1–8.	**D(0)** i\j | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 1 | 0 | 8 | 2 | 4 | 3 | ∞ | ∞ | ∞ 2 | 8 | 0 | ∞ | ∞ | 6 | 3 | ∞ | ∞ 3 | 2 | ∞ | 0 | 6 | ∞ | 7 | ∞ | 4 4 | 4 | ∞ | 6 | 0 | 1 | ∞ | ∞ | ∞ 5 | 3 | 6 | ∞ | 1 | 0 | ∞ | 4 | ∞ 6 | ∞ | 3 | 7 | ∞ | ∞ | 0 | 3 | 1 7 | ∞ | ∞ | ∞ | ∞ | 4 | 3 | 0 | 5 8 | ∞ | ∞ | 4 | ∞ | ∞ | 1 | 5 | 0

Крок 1. Проміжна вершина k = 1

Оновлення для пар i < j:

d_{23}: min(∞; 8 + 2) = 10

d_{24}: min(∞; 8 + 4) = 12

d_{35}: min(∞; 2 + 3) = 5

Симетричні елементи d_{ji} змінюються аналогічно.

D(1)

i\j	1	2	3	4	5	6	7	8
1	0	8	2	4	3	∞	∞	∞
2	8	0	10	12	6	3	∞	∞
3	2	10	0	6	5	7	∞	4
4	4	12	6	0	1	∞	∞	∞
5	3	6	5	1	0	∞	4	∞
6	∞	3	7	∞	∞	0	3	1
7	∞	∞	∞	∞	4	3	0	5
8	∞	∞	4	∞	∞	1	5	0

Крок 2. Проміжна вершина k = 2

Оновлення для пар i < j:

d_{16}: min(∞; 8 + 3) = 11

d_{46}: min(∞; 12 + 3) = 15

d_{56}: min(∞; 6 + 3) = 9

Симетричні елементи d_{ji} змінюються аналогічно.

D(2)

i\j	1	2	3	4	5	6	7	8
1	0	8	2	4	3	11	∞	∞
2	8	0	10	12	6	3	∞	∞
3	2	10	0	6	5	7	∞	4
4	4	12	6	0	1	15	∞	∞
5	3	6	5	1	0	9	4	∞
6	11	3	7	15	9	0	3	1
7	∞	∞	∞	∞	4	3	0	5
8	∞	∞	4	∞	∞	1	5	0

Крок 3. Проміжна вершина k = 3

Оновлення для пар i < j:

d_{16}: min(11; 2 + 7) = 9

d_{18}: min(∞; 2 + 4) = 6

d_{28}: min(∞; 10 + 4) = 14

d_{46}: min(15; 6 + 7) = 13

d_{48}: min(∞; 6 + 4) = 10

d_{58}: min(∞; 5 + 4) = 9

Симетричні елементи d_{ji} змінюються аналогічно.

D(3)

i\j	1	2	3	4	5	6	7	8
1	0	8	2	4	3	9	∞	6
2	8	0	10	12	6	3	∞	14
3	2	10	0	6	5	7	∞	4
4	4	12	6	0	1	13	∞	10
5	3	6	5	1	0	9	4	9
6	9	3	7	13	9	0	3	1
7	∞	∞	∞	∞	4	3	0	5
8	6	14	4	10	9	1	5	0

Крок 4. Проміжна вершина k = 4

Коротших маршрутів через вершину 4 не знайдено.

Матриця не змінюється: D(4) = D(3).

D(4)

i\j	1	2	3	4	5	6	7	8
1	0	8	2	4	3	9	∞	6
2	8	0	10	12	6	3	∞	14
3	2	10	0	6	5	7	∞	4
4	4	12	6	0	1	13	∞	10
5	3	6	5	1	0	9	4	9
6	9	3	7	13	9	0	3	1
7	∞	∞	∞	∞	4	3	0	5
8	6	14	4	10	9	1	5	0

Крок 5. Проміжна вершина k = 5

Оновлення для пар i < j:

d_{17}: min(∞; 3 + 4) = 7

d_{24}: min(12; 6 + 1) = 7

d_{27}: min(∞; 6 + 4) = 10

d_{37}: min(∞; 5 + 4) = 9

d_{46}: min(13; 1 + 9) = 10

d_{47}: min(∞; 1 + 4) = 5

Симетричні елементи d_{ji} змінюються аналогічно.

D(5)

i\j	1	2	3	4	5	6	7	8
1	0	8	2	4	3	9	7	6
2	8	0	10	7	6	3	10	14
3	2	10	0	6	5	7	9	4
4	4	7	6	0	1	10	5	10
5	3	6	5	1	0	9	4	9
6	9	3	7	10	9	0	3	1
7	7	10	9	5	4	3	0	5
8	6	14	4	10	9	1	5	0

Крок 6. Проміжна вершина k = 6

Оновлення для пар i < j:

d_{27}: min(10; 3 + 3) = 6

d_{28}: min(14; 3 + 1) = 4

d_{78}: min(5; 3 + 1) = 4

Симетричні елементи d_{ji} змінюються аналогічно.

D(6)

i\j	1	2	3	4	5	6	7	8
1	0	8	2	4	3	9	7	6
2	8	0	10	7	6	3	6	4
3	2	10	0	6	5	7	9	4
4	4	7	6	0	1	10	5	10
5	3	6	5	1	0	9	4	9
6	9	3	7	10	9	0	3	1
7	7	6	9	5	4	3	0	4
8	6	4	4	10	9	1	4	0

Крок 7. Проміжна вершина k = 7

Оновлення для пар i < j:

d_{46}: min(10; 5 + 3) = 8

d_{48}: min(10; 5 + 4) = 9

d_{56}: min(9; 4 + 3) = 7

d_{58}: min(9; 4 + 4) = 8

Симетричні елементи d_{ji} змінюються аналогічно.

D(7)

i\j	1	2	3	4	5	6	7	8
1	0	8	2	4	3	9	7	6
2	8	0	10	7	6	3	6	4
3	2	10	0	6	5	7	9	4
4	4	7	6	0	1	8	5	9
5	3	6	5	1	0	7	4	8
6	9	3	7	8	7	0	3	1
7	7	6	9	5	4	3	0	4
8	6	4	4	9	8	1	4	0

Крок 8. Проміжна вершина k = 8

Оновлення для пар i < j:

d_{16}: min(9; 6 + 1) = 7

d_{23}: min(10; 4 + 4) = 8

d_{36}: min(7; 4 + 1) = 5

d_{37}: min(9; 4 + 4) = 8

Симетричні елементи d_{ji} змінюються аналогічно.
Перший рядок D(8): 0, 8, 2, 4, 3, 7, 7, 6 — збігається з dist алгоритму Дейкстри.

D(8)

i\j	1	2	3	4	5	6	7	8
1	0	8	2	4	3	7	7	6
2	8	0	8	7	6	3	6	4
3	2	8	0	6	5	5	8	4
4	4	7	6	0	1	8	5	9
5	3	6	5	1	0	7	4	8
6	7	3	5	8	7	0	3	1
7	7	6	8	5	4	3	0	4
8	6	4	4	9	8	1	4	0

У результаті ручного виконання алгоритму Флойда (таблиця 6.4) було побудовано послідовність матриць від $D^{(0)}$ до $D^{(8)}$. На кожному кроці здійснювалася перевірка можливості скорочення відстаней між парами вершин через чергову проміжну вершину.

Після завершення восьми ітерацій отримано симетричну матрицю найкоротших відстаней між усіма вершинами графа. Усі її елементи є скінченними, що відповідає зв'язності графа.

Порівняння першого рядка кінцевої матриці $D^{(8)}$ з результатами алгоритму Дейкстри (таблиці 6.1-6.3) показало повний збіг найкоротших відстаней від вершини 1 до всіх інших вершин. Це підтверджує узгодженість результатів ручного виконання обох алгоритмів.

5. Програмна реалізація алгоритму Флойда

Для перевірки результатів ручного трасування алгоритму Флойда було розроблено програмну реалізацію мовою Python. Як вхідні дані використано зважений неорієнтований граф варіанта №15, що містить 8 вершин і 14 ребер.

Програма формує початкову матрицю ваг графа та послідовно виконує перевірку можливості скорочення відстаней між усіма парами вершин через проміжні вершини. На кожній ітерації здійснюється оновлення елементів матриці відстаней відповідно до алгоритму Флойда.

Для наочного представлення результатів програма виводить початкову матрицю $D^{(0)}$, усі проміжні матриці $D^{(1)}$–$D^{(7)}$ та кінцеву матрицю $D^{(8)}$. Також відображаються пари вершин, для яких знайдено коротші маршрути.

Python-код програмної реалізації алгоритму Флойда наведено в Лістингу 6.4.

Лістинг 6.4 – Python-код реалізації алгоритму Флойда

```
# Кількість вершин графа
N = 8

# Позначення нескінченності
INF = float("inf")
```

```
# Ребра графа варіанта №15 (u, v, вага)
EDGES = [
    (1, 2, 8), (1, 3, 2), (1, 4, 4), (1, 5, 3),
    (2, 5, 6), (2, 6, 3), (3, 4, 6), (3, 6, 7),
    (3, 8, 4), (4, 5, 1), (5, 7, 4), (6, 7, 3),
    (6, 8, 1), (7, 8, 5)
]


def build_weight_matrix(n, edges):
    """Побудова початкової матриці ваг W."""

    # Ініціалізація матриці
    W = [[INF] * n for _ in range(n)]

    # Нульові елементи головної діагоналі
    for i in range(n):
        W[i][i] = 0

    # Заповнення матриці вагами ребер
    for u, v, weight in edges:
        W[u - 1][v - 1] = weight
        W[v - 1][u - 1] = weight

    return W


def format_value(value):
    """Відображення нескінченності."""
    return "∞" if value == INF else str(value)


def display_matrix(D, k):
    """Виведення матриці відстаней D(k)."""

    print(f"\n--- Матриця D({k}) ---")

    # Заголовки стовпців
    print("     " + "".join(
        f"{j:>4}" for j in range(1, len(D) + 1)
    ))

    # Виведення рядків матриці
    for i, row in enumerate(D, start=1):
        values = "".join(
            f"{format_value(x):>4}" for x in row
        )
        print(f"{i:>3}  {values}")


def floyd(W):
    """
    Алгоритм Флойда.
    Вхідні дані: матриця ваг W.
    Вихідні дані: матриця найкоротших відстаней D.
    """

    n = len(W)

    # Копіювання початкової матриці
    D = [row[:] for row in W]

    # Початковий стан
    display_matrix(D, 0)
```

```
    # Послідовно розглядаємо проміжні вершини
    for k in range(n):

        # Список змінених відстаней
        changes = []

        # Перебираємо початкові вершини
        for i in range(n):

            # Перебираємо кінцеві вершини
            for j in range(n):

                # Перевірка досяжності через k
                if D[i][k] == INF or D[k][j] == INF:
                    continue

                # Довжина шляху через вершину k
                new_distance = D[i][k] + D[k][j]

                # Перевірка можливості скорочення
                if new_distance < D[i][j]:

                    old_distance = D[i][j]

                    # Оновлення найкоротшої відстані
                    D[i][j] = new_distance

                    # Для неорієнтованого графа
                    # виводимо кожну пару один раз
                    if i < j:
                        changes.append((
                            i + 1, j + 1,
                            old_distance, new_distance
                        ))

        # Виведення результатів поточного кроку
        print(f"\nКрок {k + 1}. "
              f"Проміжна вершина k = {k + 1}")

        if changes:
            print("Оновлення відстаней:")

            for u, v, old, new in changes:
                print(
                    f"  d[{u},{v}]: "
                    f"{format_value(old)} -> {new}"
                )
        else:
            print("Оновлень немає.")

        # Матриця після поточної ітерації
        display_matrix(D, k + 1)

    return D

# Основна частина програми
W = build_weight_matrix(N, EDGES)

print("=== Алгоритм Флойда ===")
print("Варіант №15, кількість вершин:", N)

# Побудова матриці найкоротших відстаней
D = floyd(W)
```

```
# Підсумковий результат
print("\n --- Кінцева матриця відстаней ---")
display_matrix(D, N)

# Перевірка результату алгоритму Дейкстри
dijkstra_dist = [0, 8, 2, 4, 3, 7, 7, 6]

print("\n--- Порівняння з алгоритмом Дейкстри ---")
print("Дейкстра:", dijkstra_dist)
print("Флойд:   ", D[0])

if D[0] == dijkstra_dist:
    print("Результати повністю збігаються.")
else:
    print("Виявлено розбіжності.")
```

У результаті виконання Python-коду, наведеного в Лістингу 6.4, було побудовано послідовність матриць найкоротших відстаней для графа варіанта №15. Програма виконала вісім ітерацій алгоритму Флойда, послідовно перевіряючи можливість скорочення маршрутів через проміжні вершини.

На кожному кроці було сформовано відповідну матрицю відстаней та визначено пари вершин, для яких знайдено коротші шляхи. Результати програмного виконання алгоритму Флойда наведено в таблиці 6.5.

Таблиця 6.5 – Результати виконання алгоритму Флойда мовою Python

```
=== Алгоритм Флойда ===
Варіант №15, кількість вершин: 8

--- Матриця D(0) ---
        1   2   3   4   5   6   7   8
  1     0   8   2   4   3   ∞   ∞   ∞
  2     8   0   ∞   ∞   6   3   ∞   ∞
  3     2   ∞   0   6   ∞   7   ∞   4
  4     4   ∞   6   0   1   ∞   ∞   ∞
  5     3   6   ∞   1   0   ∞   4   ∞
  6     ∞   3   7   ∞   ∞   0   3   1
  7     ∞   ∞   ∞   ∞   4   3   0   5
  8     ∞   ∞   4   ∞   ∞   1   5   0

Крок 1. Проміжна вершина k = 1
Оновлення відстаней:
  d[2,3]: ∞ -> 10
  d[2,4]: ∞ -> 12
  d[3,5]: ∞ -> 5

--- Матриця D(1) ---
        1   2   3   4   5   6   7   8
  1     0   8   2   4   3   ∞   ∞   ∞
  2     8   0  10  12   6   3   ∞   ∞
  3     2  10   0   6   5   7   ∞   4
  4     4  12   6   0   1   ∞   ∞   ∞
```

```
  5     3   6   5   1   0   ∞   4   ∞
  6     ∞   3   7   ∞   ∞   0   3   1
  7     ∞   ∞   ∞   ∞   4   3   0   5
  8     ∞   ∞   4   ∞   ∞   1   5   0

Крок 2. Проміжна вершина k = 2
Оновлення відстаней:
  d[1,6]: ∞ -> 11
  d[4,6]: ∞ -> 15
  d[5,6]: ∞ -> 9

--- Матриця D(2) ---
        1   2   3   4   5   6   7   8
  1     0   8   2   4   3  11   ∞   ∞
  2     8   0  10  12   6   3   ∞   ∞
  3     2  10   0   6   5   7   ∞   4
  4     4  12   6   0   1  15   ∞   ∞
  5     3   6   5   1   0   9   4   ∞
  6    11   3   7  15   9   0   3   1
  7     ∞   ∞   ∞   ∞   4   3   0   5
  8     ∞   ∞   4   ∞   ∞   1   5   0

Крок 3. Проміжна вершина k = 3
Оновлення відстаней:
  d[1,6]: 11 -> 9
  d[1,8]: ∞ -> 6
  d[2,8]: ∞ -> 14
  d[4,6]: 15 -> 13
  d[4,8]: ∞ -> 10
  d[5,8]: ∞ -> 9

--- Матриця D(3) ---
        1   2   3   4   5   6   7   8
  1     0   8   2   4   3   9   ∞   6
  2     8   0  10  12   6   3   ∞  14
  3     2  10   0   6   5   7   ∞   4
  4     4  12   6   0   1  13   ∞  10
  5     3   6   5   1   0   9   4   9
  6     9   3   7  13   9   0   3   1
  7     ∞   ∞   ∞   ∞   4   3   0   5
  8     6  14   4  10   9   1   5   0

Крок 4. Проміжна вершина k = 4
Оновлень немає.

--- Матриця D(4) ---
        1   2   3   4   5   6   7   8
  1     0   8   2   4   3   9   ∞   6
  2     8   0  10  12   6   3   ∞  14
  3     2  10   0   6   5   7   ∞   4
  4     4  12   6   0   1  13   ∞  10
  5     3   6   5   1   0   9   4   9
  6     9   3   7  13   9   0   3   1
  7     ∞   ∞   ∞   ∞   4   3   0   5
  8     6  14   4  10   9   1   5   0

Крок 5. Проміжна вершина k = 5
Оновлення відстаней:
  d[1,7]: ∞ -> 7
  d[2,4]: 12 -> 7
  d[2,7]: ∞ -> 10
  d[3,7]: ∞ -> 9
  d[4,6]: 13 -> 10
  d[4,7]: ∞ -> 5
```

```
--- Матриця D(5) ---
        1   2   3   4   5   6   7   8
  1     0   8   2   4   3   9   7   6
  2     8   0  10   7   6   3  10  14
  3     2  10   0   6   5   7   9   4
  4     4   7   6   0   1  10   5  10
  5     3   6   5   1   0   9   4   9
  6     9   3   7  10   9   0   3   1
  7     7  10   9   5   4   3   0   5
  8     6  14   4  10   9   1   5   0

Крок 6. Проміжна вершина k = 6
Оновлення відстаней:
  d[2,7]: 10 -> 6
  d[2,8]: 14 -> 4
  d[7,8]: 5 -> 4

--- Матриця D(6) ---
        1   2   3   4   5   6   7   8
  1     0   8   2   4   3   9   7   6
  2     8   0  10   7   6   3   6   4
  3     2  10   0   6   5   7   9   4
  4     4   7   6   0   1  10   5  10
  5     3   6   5   1   0   9   4   9
  6     9   3   7  10   9   0   3   1
  7     7   6   9   5   4   3   0   4
  8     6   4   4  10   9   1   4   0

Крок 7. Проміжна вершина k = 7
Оновлення відстаней:
  d[4,6]: 10 -> 8
  d[4,8]: 10 -> 9
  d[5,6]: 9 -> 7
  d[5,8]: 9 -> 8

--- Матриця D(7) ---
        1   2   3   4   5   6   7   8
  1     0   8   2   4   3   9   7   6
  2     8   0  10   7   6   3   6   4
  3     2  10   0   6   5   7   9   4
  4     4   7   6   0   1   8   5   9
  5     3   6   5   1   0   7   4   8
  6     9   3   7   8   7   0   3   1
  7     7   6   9   5   4   3   0   4
  8     6   4   4   9   8   1   4   0

Крок 8. Проміжна вершина k = 8
Оновлення відстаней:
  d[1,6]: 9 -> 7
  d[2,3]: 10 -> 8
  d[3,6]: 7 -> 5
  d[3,7]: 9 -> 8

--- Матриця D(8) ---
        1   2   3   4   5   6   7   8
  1     0   8   2   4   3   7   7   6
  2     8   0   8   7   6   3   6   4
  3     2   8   0   6   5   5   8   4
  4     4   7   6   0   1   8   5   9
  5     3   6   5   1   0   7   4   8
  6     7   3   5   8   7   0   3   1
  7     7   6   8   5   4   3   0   4
  8     6   4   4   9   8   1   4   0
```

```
--- Кінцева матриця відстаней ---

--- Матриця D(8) ---
        1   2   3   4   5   6   7   8
  1     0   8   2   4   3   7   7   6
  2     8   0   8   7   6   3   6   4
  3     2   8   0   6   5   5   8   4
  4     4   7   6   0   1   8   5   9
  5     3   6   5   1   0   7   4   8
  6     7   3   5   8   7   0   3   1
  7     7   6   8   5   4   3   0   4
  8     6   4   4   9   8   1   4   0

--- Порівняння з алгоритмом Дейкстри ---
Дейкстра: [0, 8, 2, 4, 3, 7, 7, 6]
Флойд:    [0, 8, 2, 4, 3, 7, 7, 6]
Результати повністю збігаються.
```

Аналіз результатів:

Порівняння результатів ручного трасування алгоритму Флойда (таблиця 6.4) та програмного виконання (таблиця 6.5) показало їх повний збіг. У процесі виконання програми було сформовано початкову матрицю $D^{(0)}$ та вісім наступних матриць відстаней $D^{(1)}$–$D^{(8)}$.

За результатами восьми ітерацій алгоритму було виконано 29 оновлень відстаней між різними парами вершин без урахування симетричних повторень. На четвертій ітерації скорочення відстаней не відбувалося, тоді як на інших кроках було знайдено коротші маршрути.

Отримана кінцева матриця $D^{(8)}$ містить найкоротші відстані між усіма парами вершин графа варіанта №15. Порівняння її першого рядка з результатами алгоритму Дейкстри показало повний збіг: [0, 8, 2, 4, 3, 7, 7, 6].

Таким чином, програмна реалізація алгоритму Флойда підтвердила правильність ручних розрахунків і дозволила визначити найкоротші відстані між усіма вершинами досліджуваного графа.

6. Порівняльний аналіз алгоритмів Дейкстри та Флойда

Для порівняння ефективності алгоритмів Дейкстри (Dijkstra) та Флойда (Floyd–Warshall) було проаналізовано результати їх ручного трасування та програмної реалізації мовою Python.

Обидва алгоритми використовуються для пошуку найкоротших шляхів у зважених графах, проте відрізняються принципом роботи та обсягом отриманих результатів. Алгоритм Дейкстри визначає найкоротші шляхи від однієї початкової вершини до всіх інших, тоді як алгоритм Флойда знаходить найкоротші відстані між усіма парами вершин.

Для оцінювання алгоритмів використано отримані найкоротші відстані, кількість основних ітерацій, теоретичну часову та просторову складність. Результати порівняння наведено в таблиці 6.6.

Таблиця 6.6 – Порівняльна характеристика алгоритмів Дейкстри та Флойда

Характеристика	Алгоритм Дейкстри	Алгоритм Флойда
Призначення	Найкоротші шляхи від однієї вершини	Найкоротші відстані між усіма парами вершин
Кількість вершин	8	8
Кількість ребер	14	14
Початкові дані	Список суміжності	Матриця ваг
Основний принцип	Вибір вершини з найменшою поточною відстанню та релаксація ребер	Послідовне уточнення відстаней через проміжні вершини
Кількість основних кроків	8 оброблених вершин	8 ітерацій за проміжними вершинами
Часова складність	$O((V + E) \log V)$	$O(V^3)$
Просторова складність	$O(V + E)$	$O(V^2)$
Результат	Масиви dist, pred	Матриця відстаней $D^{(8)}$

Таблиця 6.7 – Порівняння найкоротших відстаней від вершини 1

Кінцева вершина	Дейкстра	Флойд	Збіг
1	0	0	Так
2	8	8	Так
3	2	2	Так
4	4	4	Так
5	3	3	Так
6	7	7	Так
7	7	7	Так
8	6	6	Так

Отже:

$$dist_{\text{Dijkstra}} = [0, 8, 2, 4, 3, 7, 7, 6]$$

$$D_{\text{Floyd}}{}^{(8)}[1,*] = [0, 8, 2, 4, 3, 7, 7, 6]$$

$$dist_{\text{Dijkstra}} = D_{\text{Floyd}}{}^{(8)}[1,*]$$

Результати повністю збігаються для всіх восьми вершин.

Аналіз результатів:

За результатами порівняння (таблиці 6.6 і 6.7) встановлено, що алгоритми Дейкстри та Флойда забезпечили однакові найкоротші відстані від вершини 1 до всіх інших вершин графа. Результати ручного трасування повністю збіглися з результатами програмного виконання обох алгоритмів.

Алгоритм Дейкстри за допомогою черги з пріоритетом послідовно обробив 8 вершин, сформував масив найкоротших відстаней і визначив попередників для відновлення маршрутів. Алгоритм Флойда виконав 8 ітерацій за проміжними вершинами та сформував матрицю найкоротших відстаней між усіма парами вершин. Під час його виконання відбулося 29 скорочень відстаней між різними неупорядкованими парами вершин.

Теоретична часова складність алгоритму Дейкстри для використаної реалізації з чергою пріоритетів становить $O((V + E) \log V)$, тоді як алгоритм Флойда має складність $O(V^3)$. Тому для пошуку найкоротших шляхів від однієї вершини в розріджених графах доцільно використовувати алгоритм Дейкстри. Якщо

необхідно визначити найкоротші відстані між усіма парами вершин, алгоритм Флойда є зручним завдяки матричному представленню результатів.

Таким чином, обидва алгоритми правильно розв'язали поставлені задачі, а їх вибір залежить від необхідного обсягу результатів, структури графа та вимог до обчислювальної ефективності.

Висновок:

Під час виконання лабораторної роботи було досліджено, реалізовано, протестовано та проаналізовано алгоритми Дейкстри (Dijkstra) та Флойда (Floyd–Warshall) для знаходження найкоротших шляхів у зважених графах. Для дослідження використано неорієнтований зв'язний граф варіанта №15, який містить 8 вершин і 14 ребер із додатними вагами.

Шляхом ручного трасування та графічної візуалізації досліджено роботу алгоритму Дейкстри. Для початкової вершини 1 було визначено найкоротші шляхи до всіх інших вершин графа. У результаті отримано масив відстаней dist = [0, 8, 2, 4, 3, 7, 7, 6] та відповідний масив попередників pred = [-1, 1, 1, 1, 1, 8, 5, 3]. Розроблена програмна реалізація мовою Python підтвердила правильність ручних розрахунків.

За алгоритмом Флойда було виконано послідовне формування матриць відстаней $D^{(0)}$–$D^{(8)}$. Протягом восьми ітерацій відбулося 29 оновлень відстаней між різними парами вершин без урахування симетричних повторень. Отримана кінцева матриця містить найкоротші відстані між усіма вершинами графа. Результати програмної реалізації алгоритму Флойда повністю збіглися з результатами ручного трасування.

Порівняльний аналіз показав, що обидва алгоритми забезпечили однакові найкоротші відстані від вершини 1 до решти вершин графа. Водночас алгоритм Дейкстри доцільно використовувати для пошуку найкоротших шляхів від однієї заданої вершини, тоді як алгоритм Флойда дозволяє визначити найкоротші відстані

між усіма парами вершин. Для реалізації Дейкстри з чергою пріоритетів часова складність становить O((V + E) log V), а для алгоритму Флойда – $O(V^3)$.

Таким чином, у ході лабораторної роботи було набуто практичних навичок представлення зважених графів, ручного трасування алгоритмів, побудови матриць найкоротших відстаней, програмної реалізації та перевірки правильності отриманих результатів. Дослідження підтвердило ефективність обох алгоритмів для розв'язання відповідних задач пошуку найкоротших шляхів.

Посилання на GitHub:

Посилання на онлайн компілятор: https://coddy.tech/playground/ru/python